LLM Coaching App -
https://github.com/KaurArshnoor/LLM_coach_app

Author: Arshnoor Kaur

Overview

This app implements an LLM coaching system that converts user queries into exercise recommendations. It uses a two-stage pipeline: retrieval (SQL) and LLM-based re-ranking.

Total time: 6.5 hours

- Data & DB Setup (1 hrs): Schema design, PostgreSQL setup, and CSV ingestion script.
- Retrieval Logic (1.5 hr): Implementing keyword-based SQL search and ranking fallbacks.
- LLM Pipeline (1.5 hrs): Groq API integration, prompt engineering for JSON output, and re-ranking logic.
- Frontend & API (1.5 hr): FastAPI endpoints and React interface with Voice/Web Speech API.
- Documentation & System Design (1 hr): README and scaling analysis.

Data Setup

Database Initialization

I used PostgreSQL as the primary database because of its strong support for structured querying, compatibility with pgvector for embedding-based retrieval and scalability for production systems.

Data Loading

- The exercise CSV is ingested via load_data.py file
- Data is stored in a structured relational format
- Queries are executed using psycopg2

Additional Tables Added

- query_logs: stores queries, retrieved candidates, ranked outputs, and latency for analytics
- user_profiles: supports onboarding and personalization

Pipeline Design

Stage 1: Retrieval (Candidate Generation)

The system performs an ILIKE search across titles, descriptions, and tags. This narrows down the universe of exercises to the top 20 candidates.

Design Rationale:

- Fast (less than 100ms)
- Deterministic and interpretable

- Provides a strong baseline before LLM reasoning

Stage 2: LLM Re-ranking

We then pass those 20 candidates to the Llama-3.3-70b model via Groq. The LLM acts as the "brain," understanding the semantic nuance (e.g., knowing that "winger" implies a need for "lateral agility" and "explosiveness").

Backend Design

User Query -> FastAPI -> Engine

Core Components:

FastAPI Layer (app.py)

- Handles HTTP requests
- Validates input using Pydantic
- Calls engine methods
- Endpoints: /recommend, /health, /users/onboard

Engine (engine.py)

1. retrieve()
    - SQL keyword search
    - Returns candidate set
2. rank_with_grok()
    - LLM prompt construction
    - API call
    - JSON parsing
3. recommend()
    - Orchestrates full pipeline
    - Logs results

Voice Integration: Used the Web Speech API in the React frontend to allow for hands-free queries, catering to the "coaching" use case where users might be in a gym environment.

Scaling Strategy

To grow from a prototype to a production system handling 100k+ exercises and simultaneous users:

- **From Keywords to Vectors:** I would replace the ILIKE search with Vector Embeddings using pgvector. This would allow for semantic retrieval such as finding "rehab" exercises even if the word "rehab" isn't in the text.
- **Caching:** Popular queries (e.g., "leg day") should be cached in Redis to bypass the LLM call and reduce latency/cost.
- **Batching & Async Workers:** LLM calls are I/O bound. Using Celery or FastAPI's background tasks ensures the main thread isn't blocked during the 1-3s ranking window.

- **Horizontal Scaling:** Containerizing the app with Docker (as included in my repo) would allow us to spin up multiple instances behind a Load Balancer to handle concurrent users.

Creativity: Personalization via Onboarding

To move beyond generic search, I would implement a "Profile-Aware" ranking system.

Data Collection

During onboarding, we would collect:

- Physical Profile: Height, weight, and current injury list ("lower back sensitivity").
- Equipment Access: "Full gym" vs. "Home/Bodyweight only."
- Style Preference: High-intensity (HIIT) vs. slow/controlled (Pilates).

Influence on Pipeline

- Hard Constraints (Retrieval): If a user lists a "Knee Injury" the retrieval step will automatically append AND impact_level = 'low' to the SQL query.
- Soft Constraints (Ranking): The user's goal (such as “Hypertrophy”) is passed into the LLM prompt: *"Prioritize exercises that maximize time under tension for this user whose goal is Hypertrophy."*

Inspiration

- Spotify: Similar to "Daily Mixes," we could use Collaborative Filtering. If users with "Knee Pain" consistently click on "Box Squats" and ignore "Lunges," the system should learn to boost Box Squats for that cohort.
- Netflix: Using "Match %" scores based on how well the exercise tags align with the user's historical preferences and onboarding constraints.